

Commodore VIC-20 Technical Analysis

Architecture and System Design

CPU and Processor Architecture: The VIC-20 is powered by a MOS Technology 6502 microprocessor running at about 1.02 MHz (NTSC models) or 1.108 MHz (PAL models) ¹. This 8-bit CPU has a 16-bit address bus, allowing up to 64 KB of addressable memory. In the VIC-20, the 6502 is a simple in-order CISC processor with no specialized co-processors or pipelining – essentially all computation and I/O coordination is handled by the CPU itself. By modern standards the clock speed is extremely low, but the 6502's efficient design (few cycles per instruction on average) made it competitive with other early-1980s CPUs. For instance, a 1 MHz 6502 could execute roughly **0.43 MIPS** (million instructions per second) ², which is comparable to a ~3–4 MHz Zilog Z80 in many tasks ³. (In fact, contemporary 8-bit rivals like the ZX Spectrum used a 3.5 MHz Z80 and achieved similar throughput to the VIC-20's 1 MHz 6502.) Modern CPUs, in contrast, are **thousands of times faster** – a single 3 GHz core can perform on the order of 3,000 MIPS – highlighting the simplicity of the VIC-20's processor. The 6502 in the VIC-20 lacks advanced features like caches or branch prediction; however, it does support useful indexed addressing modes and an interrupt system which the VIC-20 uses for timing (e.g. generating a periodic system interrupt at 50/60 Hz via a VIA chip, discussed below).

Memory Layout and Constraints: The VIC-20's memory map is constrained by its cost-sensitive design. It includes **5 KB of static RAM** built-in (of which about 3.5 KB is free for user BASIC programs by default) ⁴. This RAM is extremely limited even by 1980s standards – for comparison, the Apple II had 16–48 KB RAM, and even today's simple microcontrollers typically have more. The VIC-20's ROM is 20 KB in total, divided into an 8 KB BASIC interpreter, an 8 KB KERNAL (OS routines) and a 4 KB character generator ROM ⁵. These ROMs occupy the upper part of the 6502 address space (for example, the BASIC ROM is mapped at \$C000–\$DFFF and the KERNAL at \$E000–\$FFFF) ⁶. The 5 KB RAM occupies the lower addresses; notably, some of this RAM is used for the video display buffer and system variables, leaving the ~3.5 KB free for BASIC programs ⁴. The VIC-20's architecture allows **memory expansion through the cartridge port**: up to 27.5 KB additional RAM can be added for a maximum of 32 KB total RAM (about 27.5 KB free to BASIC) ⁴. These expansion cartridges map RAM into vacant address blocks in the 6502 address space (typically in 3 KB, 8 KB or 16 KB segments). The memory map is segmented into blocks (BLK1–BLK5 in Commodore terminology) to accommodate optional expansions. For instance, with an 8 KB expansion, the BASIC program area moves to a lower address (e.g. from \$1200 to \$0400) to utilize the extra RAM. Despite the expansions, the VIC-20's memory remains very limited – a stark comparison is that **modern CPUs have L1 caches larger than the VIC-20's entire RAM**. Developers at the time had to work within these tight constraints, often using machine code or creative tricks to fit programs into a few kilobytes.

Graphics and Display (VIC Chip): The graphics and sound are handled by the MOS “VIC” (Video Interface Controller) chip, specifically the 6560 (NTSC) or 6561 (PAL) variant ⁷ ⁸. This chip was originally designed for low-cost color graphics and it defines the VIC-20's visual capabilities. The display is character-based by default: **22 columns by 23 rows** of text characters on screen ⁷. (The VIC-20's screen is notably narrower than the 40-column displays of contemporaries like the Apple II or Commodore 64, a cost trade-off that simplified the video circuitry and memory usage.) Each text character is an 8×8 pixel glyph from the built-in

character ROM (with two character sets available: one with uppercase+graphics symbols, and one with upper+lowercase). Using these characters, the VIC-20 can display simple block graphics and text with **8 selectable text colors**, and it supports a **high-resolution bitmap mode of 176×184 pixels** ⁷. In high-res mode, pixels are monochrome per 8×8 block (with a global background color), but programmers often used custom character redefinition to simulate graphics by redefining the 8×8 character patterns. There is also a rudimentary multicolor mode, effectively halving horizontal resolution to **88×184 pixels** but allowing up to 4 colors in a block ⁸. The color palette includes 16 background colors and 8 foreground colors ⁷, though only a subset can be used simultaneously in certain modes. The VIC chip does **not support hardware sprites or tile-scrolling** like the later VIC-II in the Commodore 64 – all moving graphics had to be done by redrawing character patterns or screen memory in software, which was CPU-intensive. The VIC-20 does have a light pen input line (available via the user port) and could detect a light pen trigger on the raster beam ⁹.

The VIC chip also generates **sound for the VIC-20**. It provides **3 square-wave audio channels** (voices) plus one noise generator ¹⁰ ¹¹. Each voice has programmable frequency (covering roughly 5 octaves, about 128 discrete notes) and a shared volume control. Unlike the more sophisticated SID chip in the Commodore 64, the VIC's sound is simple – there are no hardware envelopes or filters, just basic tone generation. Still, it can produce music and sound effects typical of early 1980s games (beeps, simple melodies, noise for explosions, etc.). The sound output is combined with the video signal (for RF/composite output). **Graphics and sound share memory with the CPU**, meaning the VIC chip reads character and color data from the main RAM. Notably, the VIC-20's use of **static RAM** and the relatively low screen resolution mean the VIC chip's memory access demands are modest compared to later systems. The 6560/6561 fetches video data during the active video scan, and in those moments the 6502 may be briefly halted to avoid bus conflicts (similar to how the C64's VIC-II "steals" cycles) ¹². However, because the VIC-20 has fewer columns/rows, the CPU loses fewer cycles to video refreshing than it would in a 40-column display; when the screen is blanked, the CPU can use nearly the full bus bandwidth. In modern terms, one can liken the VIC-20's video system to a very basic GPU that *directly* shares memory with the CPU – there's no frame buffer abstraction or GPU memory, just direct reading of a section of RAM for video. This tight coupling meant that higher display detail directly reduced CPU time available (a trade-off somewhat analogous to how integrated graphics today can eat into system RAM and memory bandwidth).

Motherboard and I/O Subsystems: Architecturally, the VIC-20's motherboard consists of the 6502 CPU, the VIC chip, two VIA chips, and the ROM/RAM chips all interconnected on a simple bus. The **system bus** is the 8-bit data bus and 16-bit address bus of the 6502, running at roughly 1 MHz. The two **6522 VIA (Versatile Interface Adapter) chips** are crucial to the I/O architecture ¹³. These VIA chips provide general-purpose I/O ports, timers, and handshake lines. Commodore used them to interface with practically all peripherals: - **VIA #2** (at memory address \$9120–\$912F) handles the **keyboard matrix scanning** and some joystick and cassette lines. Its 16-bit parallel ports are configured such that one port drives the keyboard column lines and the other reads keyboard row lines (a classic row/column keyboard matrix) ¹⁴. VIA #2 also has a line for the cassette motor control and cassette read/write signals, as well as one joystick input bit (the VIC-20 has one digital joystick port, which electrically is wired partly into the VIA) ¹⁴. This VIA also supplies the periodic **interrupt (IRQ) timer** for the system: it can be set to generate interrupts at 60 Hz (NTSC) or 50 Hz (PAL) for system timing using Timer 1 ¹⁵. - **VIA #1** (at \$9110–\$911F) is dedicated to other I/O and expansion. Notably, one 8-bit port of VIA#1 is completely devoted to the **user port interface** (providing 8 data lines on that connector) and its two handshake lines (CB1/CB2) ¹⁶ ¹⁷. The other port on VIA#1 is used for a mix of serial bus signals and joystick/light pen inputs ¹⁸. Specifically, bits of VIA#1 Port A and some control lines tie into the **Commodore serial IEEE-488 bus** (the 6-pin DIN connector) – signals for

Serial ATN (Attention), Clock, Data, etc., are managed by VIA lines ¹⁸ ¹⁹ . Additionally, VIA#1 handles the **joystick inputs** (three directional bits from the joystick plus the fire button/light-pen sense) ²⁰ , and the cassette “sense” input (detecting the play button via a switch) ²¹ . In summary, **all VIC-20 I/O – keyboard, joystick, tape, disk serial bus, user port – is managed by the two VIA chips**. Commodore chose to use these VIA interface chips (which they produced in-house at MOS Technology) rather than simpler TTL logic, because they provided a flexible, programmable way to handle I/O lines ²² ²³ . This made the design *clean* and similar to the earlier PET computer architecture which also used VIA/PIA chips ²⁴ . (By comparison, some competitors like Sinclair did minimal I/O chips to cut cost, but Commodore embraced VIA chips for versatility.)

System Bus and Expansion: The VIC-20 provides several expansion ports that directly tap into the system bus or VIA I/O lines: - A **44-pin edge Cartridge/Expansion port on the back exposes the 6502 address and data buses, power rails, and control signals** ²⁵ ²⁶ . **This allows ROM cartridges or RAM expansions to be mapped into the address space. It essentially makes the VIC-20 a backplane for add-ons, similar to how modern microcontrollers use shield connectors. Cartridges could contain game ROMs, additional RAM, or even hardware expansions (e.g. one could add an IEEE-488 parallel disk interface via a cartridge).** The port decoding supports plug-in memory at specific address blocks (e.g. an 8 KB ROM would typically occupy \$A000–\$BFFF). The system bus operates at ~1 MHz, so expansion devices had to match that speed. In modern terms, this is akin to an extremely simple ISA bus – unbuffered and without plug-and-play, but very direct. - A GEI (Commodore) Serial IEEE-488 bus **port (a circular 6-pin DIN) is provided for connecting disk drives and printers** ²⁷ . **This is the same serial bus used later on the Commodore 64 and other CBM machines, allowing the VIC-20 to use peripherals like the VIC-1540 disk drive or 1525 printer.** The bus is *serial* (one data line, one clock line, plus handshake lines), which was a cost-saving alternative to the parallel IEEE-488 used on the Commodore PET. However, this serial bus was relatively slow and CPU-intensive (since the 6522 VIA had to bit-bang the lines in software). In fact, the VIA's shift-register hardware for serial had a known bug, so Commodore's ROM firmware fell back to software-driven serial transfer, yielding a throughput of only a few hundred bytes per second in stock form – a notable I/O bottleneck of the VIC-20. **Disk operations thus felt quite sluggish (e.g. loading a large program could take tens of seconds). This was later mitigated by third-party fast loader software and cartridges.** - Video output **is through a composite video jack (or RF modulator to TV).** The VIC-20 has a 5-pin DIN for A/V: it supplies composite video and audio output, and in some models an external RF modulator could be powered via a 6 V pin ²⁸ . There is no separate high-frequency clock or GPU memory – the VIC chip drives video directly from main RAM as described. - Joystick port: **The VIC-20 includes one DE-9 game controller port on the side** ²⁹ . This port is electrically similar to the Atari-standard joystick port: it has 5 digital inputs (4 directions + 1 button) read by the VIA, and also two analog potentiometer inputs (which on VIC-20 are not used since the VIC chip doesn't have paddle ADC like the C64's SID – so paddles were not supported on VIC-20). The joystick port shares some signals with the user port and VIA, for example, the joystick button is the same line as the light pen input and goes to VIA#1 CA2 ⁹ . The limited number of joystick ports (just one on the VIC-20) meant two-player games required taking turns or using the keyboard for a second player, unlike systems with two ports. - Cassette port: **A 12-pin edge connector for the Commodore Datasette tape drive is present (the VIC-20 used the same cassette units as the PET and later C64)** ³⁰ . The cassette interface is digital (the VIA toggles a write line to save data and reads a digital pulse train for input) and runs at ~300 baud for loading programs – that's about 30 bytes per second, meaning a full load of the 5 KB memory could take a couple of minutes. **This was a very slow storage method, but common for the era. The VIA also controls the cassette motor on/off and senses the tape's play/stop button.** - Power and others:** The VIC-20's early models used an external

9VAC transformer feeding a simple internal regulator (2-pin connector), whereas later cost-reduced models used the same 7-pin power DIN as the C64 (providing +5 VDC and 9 VAC) ³¹ ³² . The inclusion of AC was to run the VIC chip's internal clock circuit and provide ~12 V for the cassette motor. The system has a built-in RESET circuit and also a reset pin exposed on the user port (useful for expansions or an external reset button).

In summary, the VIC-20's architecture is a **textbook 8-bit home computer**: a single 6502 CPU driving everything, memory mapped I/O through versatile interface adapters, and a unified bus for CPU, video, and peripherals. It's a much simpler architecture than modern PCs (which have multiple busses, DMA controllers, etc.), but it laid the groundwork. A modern computer engineer would recognize in the VIC-20 the fundamentals of **memory-mapped devices**, **interrupt-driven I/O**, and trade-offs between cost and capability. In a sense, the VIC-20 is closer to today's microcontroller-based embedded systems than to a modern PC – everything is on one chip or a few chips, and the programmer often directly twiddles I/O registers to control hardware.

Performance Characteristics

Speed Compared to Peer Systems: Despite its simplicity, the VIC-20's performance can be quantified relative to other early 1980s computers. The 6502 CPU at ~1 MHz executes most instructions in 2–4 clock cycles, giving an average throughput around 0.3–0.5 million instructions per second ² . This was roughly on par with the Apple II (1 MHz 6502) and not far behind the more expensive **BBC Micro** (which had a 2 MHz 6502, roughly doubling integer performance). Against Z80-based computers, the VIC-20 held its own in many cases: a 4 MHz Z80 (as in the later ZX Spectrum or Amstrad CPC) can retire about 0.58 MIPS ² , so the VIC-20's 0.43 MIPS was equivalent to a Z80 running ~3 MHz in typical code. That said, specific performance could vary – the Z80 had an edge in 16-bit arithmetic, whereas the 6502 excelled in memory-indexing and branching. Compared to the **Atari 8-bit** computers (e.g. Atari 800) which ran a 6502 at 1.79 MHz, the VIC-20's raw CPU was slower (roughly half the speed), but Atari's system also dedicated some CPU time to managing its ANTIC graphics co-processor. The **Commodore 64**, released a year after the VIC-20, actually had the *same 1 MHz CPU speed*, so pure computation on a VIC-20 is similar to a stock C64. However, the C64's more advanced VIC-II graphics chip steals more cycles for video DMA and the C64's use of DRAM required memory-refresh cycles, meaning the **VIC-20 sometimes could execute a bit more freely** during each frame. (For example, on a C64 the CPU effectively gets only about 75% of the cycles when the screen is on; the VIC-20's simpler display gave the CPU a relatively higher share of cycles – there were fewer “bad lines” of video where the CPU is paused, simply because the screen had fewer characters to fetch.) In practice, a C64 still outperformed VIC-20 in any task that needed more memory or better graphics, but for number-crunching or logic, the speeds were in the same ballpark.

Throughput and Bottlenecks: The VIC-20's performance bottlenecks were usually not the CPU core itself, but the limited memory and slow I/O: - *Memory Bottleneck:* With only 5 KB RAM, algorithms had to be very memory-efficient. This often meant using 8-bit arithmetic and avoiding large data structures. The limited RAM also impacts speed because it constrained buffering – e.g. you could not load large assets to process in memory all at once. (Modern analogy: it's like trying to run programs with only CPU registers and a tiny scratchpad, with no room for caching data – today even small microcontrollers often have more RAM than the VIC-20 had.) - *Video Contention:* The VIC chip and CPU share the bus, which means when the VIC chip reads screen data or color data, the CPU must wait. The 6560 VIC accesses memory primarily during the visible scan lines. There is no *separate video RAM*, so these conflicts are inevitable. Fortunately, because the VIC-20's resolution and refresh are modest (e.g., 22×23 chars @ ~60 Hz NTSC), the overhead is limited. If a

program didn't need the display, it could even disable video output to gain extra CPU cycles (blanking the screen to let the CPU use nearly 100% cycles, a trick similar to what C64 programmers did to speed up routines) ¹². The latency of accessing RAM was on the order of 1 microsecond per byte (since the bus is ~1 MHz), and that applies to both CPU and VIC fetches. Compared to modern systems where memory latency is measured in nanoseconds (and mitigated by caches), the VIC-20's CPU spent a significant fraction of time just waiting on memory accesses or video refresh. However, since everything ran at the same slow speed, this was the normal state of affairs and well-balanced in design.

- *I/O Throughput: Disk and tape I/O were very slow* on the VIC-20. The IEC serial bus for floppy drives could manage only a few hundred bytes per second with the default routines – for example, loading an 8 KB cartridge game from disk could take ~30 seconds. This was a known bottleneck (the cause was partly a VIA serial shift-register bug that forced bit-by-bit transfer under software timing). Many users relied on cartridge software or fastloaders which improved throughput somewhat (by more efficient transfer protocols), but the fundamental limit was the serial clock rate and CPU handshaking. **Cassette tape** was even slower: around 300 baud (bits per second), or ~30 bytes/s. To load just a 3 KB BASIC program might take over a minute from tape. In contrast, other computers of the era had a mix of speeds: the Apple II's disk (with a dedicated controller card) was much faster (~15 KB/s), whereas tape-based systems (Sinclair, etc.) were similarly slow or slower. Thus, the VIC-20 was **I/O-bound** when it came to data loading. From a modern perspective, the VIC-20's I/O is glacial – even a typical 2020s internet connection can transfer in one second what would have taken the VIC-20 several hours over its serial bus.

- *Graphics and Sound Performance*: Because there is no hardware acceleration for graphics, any game or graphical application must use CPU time to draw or move objects on screen. For example, to animate a simple sprite, one might redraw characters or pixel blocks each frame. At ~60 FPS, the CPU has only ~16.7 ms per frame (approximately 17,000 CPU cycles on NTSC) to update the screen and do game logic. Drawing a full screen (around 400 characters) could consume a large portion of that frame time, so VIC-20 games often had to use tiny graphics or minimal on-screen movement to maintain performance. The limited resolution actually helped here – fewer pixels to manipulate means less work. Sound generation on the VIC-20 did not significantly burden the CPU because once you set the registers for the 3 voices, the VIC chip produced sound autonomously. The CPU only updates sound registers when changing notes or volume. So sound was not a bottleneck except that complex music playback routines would steal some CPU time (especially if done via software waveforms, which was rare on VIC-20 due to limited capability).

- *Comparative Capability*: In terms of **computational tasks**, the VIC-20 can compute simple math, text processing, and game logic reasonably well for its class. A common benchmark of the era was how fast it could run a BASIC loop or simple arithmetic. VIC-20's BASIC was fairly slow (being an interpreter and with no math hardware), e.g. it could execute on the order of only a few hundred BASIC statements per second. Machine language was much faster and was used for performance-critical software. Still, any kind of heavy computation (say, calculating primes or doing floating-point math) would be extremely slow relative to modern expectations. Floating point in BASIC was software-based 40-bit precision and could take thousands of cycles per operation. A modern comparison might be: a single calculation that your smartphone does in a fraction of a microsecond might take the VIC-20 a full second of number crunching. For interactive uses like word processing or simple games, the VIC-20 was adequate, but it struggled with tasks like 3D graphics, complex sound synthesis, or large dataset handling – tasks trivial for modern PCs or even smartphones.

Latency Considerations: In the VIC-20's design, *latency* mainly comes into play with I/O devices:

- **Keyboard latency**: The keyboard is scanned by the VIA in an interrupt-driven manner (60 Hz IRQ driving a scan routine) or by tight loops. The response is effectively instantaneous to a human – a keypress would be read within at most 1/60th of a second. Modern computers poll keyboards via USB in roughly the same timeframe (every 8 ms or so), so in that sense the VIC-20's keyboard latency isn't bad at all.
- **Display refresh**

and response: The VIC-20's CRT output refreshes ~60 (NTSC) or 50 (PAL) times per second. This means any graphical change or user feedback on screen could be at most one frame (~20 ms) behind the CPU action. Input-to-output latency (e.g., pressing a key and seeing a character echo on screen) was largely governed by the 60 Hz loop and the BASIC interpreter speed, but typically was on the order of tens of milliseconds – perceptible but minor. Modern systems aiming for 60 FPS have similar frame times, though modern GUIs can add additional buffering latency. The VIC-20, being simple, has essentially no buffering – what the CPU writes to video memory appears on the next frame drawn. - **Serial communication latency:** The user port or serial port when used for modems (e.g., the VICModem at 300 baud) had significant latency if measured in absolute terms, because at 300 baud each byte takes ~.03 seconds. But in context, that was acceptable for text transmission of the day. Modern networks are thousands of times faster, making the VIC-20's serial feel extremely slow. However, one positive aspect is that the VIC-20, having direct control of I/O lines, could be very deterministic – a bit could be toggled at precise intervals (to the microsecond scale) if needed. For example, the bit-banging of the serial port was timing-critical and the 6502 could reliably generate those timings.

In summary, the VIC-20's performance is modest even by 1980s standards, oriented toward basic home computing tasks. It was **outperformed by later 8-bit machines** (the C64, BBC Micro, Sinclair Spectrum all either had more RAM, faster CPU or better graphics), but it brought a balance of capabilities at low cost. From a modern perspective, it operates in the realm of microcontroller-like speeds and memory sizes. Indeed, many of today's **embedded systems (like an Arduino Uno with an 8-bit AVR at 16 MHz, 2 KB RAM)** are in a similar spirit – they can blink LEDs, read sensors, and even generate simple video or sound, but any heavy computation must be approached cleverly or is outright impossible due to resource limits. The VIC-20, as a pioneering home computer, taught programmers to work within severe constraints, a practice still very relevant in embedded computing today.

The User Port: Pinout, Protocols, and Applications

One distinctive feature of the VIC-20 (and other Commodore 8-bit machines) is its **user port** – a general-purpose I/O expansion connector. The VIC-20's user port is a **24-pin edge connector** (often a card-edge slot at the back of the unit) providing access to the 6522 VIA #1's Port B lines and some additional signals ¹⁷. It's essentially an early forerunner of today's GPIO headers on microcontroller boards, allowing the user to interface the computer with external hardware. Below, we break down the details of this port:

Pinout and Electrical Characteristics: The user port has 12 contacts on the top side and 12 on the bottom (often labeled 1–12 on one side and A–N on the other, skipping I to avoid confusion with 1) ³³ ³⁴. Not all pins are used for digital I/O; some carry power or other special signals. Key pins include:

- **Power and Ground:** Pin 2 on the user port provides +5 V DC power (limited to 100 mA draw) ³³, and there are multiple ground pins (to ensure a common reference and allow higher current return) ³³. The VIC-20 user port also interestingly brings out the 9 VAC lines from the transformer on two pins (pins 10 and 11) ³⁵ – this AC was seldom used by peripherals, but it was present (for example, could drive modest external circuitry or timing circuits). The +5 V and ground allow simple external devices to be powered directly from the VIC-20, though with the 100 mA current cap, one could only run small circuits (modern comparison: it's similar to the 5 V pin on a Raspberry Pi's GPIO which can power external modules, but must be used carefully to avoid overdrawing).

- **Reset and Control:** Pin 3 is the **/RESET line** ³⁶. Grounding this pin will reset the VIC-20 (cold start). It effectively parallels the reset button internally and can also act as an output – it goes low when the VIC-20 resets, so external devices can be reset in tandem. This was useful for resetting expansion hardware or implementing an external reset button.
- **Joystick and Serial shared lines:** Pins 4, 5, 6 on the user port correspond to **JOYSTICK inputs (JOY0, JOY1, JOY2)** ³⁷. These are actually the same signals used by the main joystick port (for up, down, left for the analog paddles, etc., or general purpose inputs). Commodore provided them on the user port presumably for compatibility with older PET peripherals (the PET used the user port for game paddles) or simply to allow monitoring of joystick state from an expansion. Pin 7 is the **light pen / joystick fire** input (Llightpen) ³⁸. Pin 8 is the **cassette switch sense** line ³⁹ (this line goes low when the Datasette's play button is pressed – basically a “cassette ready” signal into VIA). And pin 9 is **Serial ATN IN** ⁴⁰ – this taps into the IEC serial bus Attention line. The presence of these signals on the user port means the user port can actually monitor or even override some of the VIC-20's other I/O lines if needed. (Notably, because the VIC-20's user port and other ports share the VIA, one must be careful: for example, using the user port to drive JOY0 could interfere with an actual joystick input, etc. In practice, most user port projects avoid those shared lines unless intentionally interacting with those devices.)
- **Primary User I/O Lines (Port B):** The remaining pins on the bottom side (labeled C, D, E, F, H, J, K, L in documentation) correspond to the **8-bit parallel I/O lines** of VIA #1 Port B ¹⁷. These are the core of the user port's digital I/O capability. *Each of these 8 lines (PB0–PB7) can be individually programmed as input or output via the VIA's Data Direction Register, and read/written via the port register* ¹⁷. In the VIC-20's memory map, Port B is at address 37136 (\$9110) and its data-direction register at 37138 (\$9112) ⁴¹. From the user's perspective (in BASIC or machine code), you could POKE values to 37138 to set a bit as output (bit=1) or input (bit=0), and then PEEK/POKE 37136 to read or write the level on those pins ¹⁷. Electrically, these lines are TTL-level (0 to +5 V) digital signals driven directly by the 6522 VIA chip. They can source or sink only a limited current (on the order of a few milliamps per line safely), so interfacing to higher-current devices often requires a buffer or driver transistor. The user manual cautioned against drawing too much current or directly driving things like motors from these lines. In essence, these 8 lines give the VIC-20 the ability to sense and control external circuits – much like a modern Arduino's GPIO pins, albeit without analog input capability.
- **Handshake / Control Lines (CB1, CB2):** In addition to the 8 port lines, the user port provides two special control lines from the VIA: **CB1 and CB2** (on pins labeled B and M respectively on the connector) ³⁴ ⁴². These are auxiliary lines that can be used as timers, interrupts or serial shift signals by the VIA. On the VIC-20, Commodore repurposed these for serial communications: **CB2 is configured as a serial output (TX line)** and **CB1 as a serial input control (often used as a ring indicator or input handshake)** ⁴³. In fact, the user port's pin assignments were chosen to line up with standard RS-232 serial signals: for example, PB0 is “RxD” (receive data), PB1 is “RTS” (request to send), PB2 “DTR” (data terminal ready), PB6 “CTS”, PB7 “DSR”, etc., and **CB2 is the transmit data (TxD) line** ⁴⁴ ⁴⁵. This effectively means the VIC-20's user port **can function as an RS-232 serial port**, albeit at TTL voltage levels and requiring software to drive the protocol. Commodore sold an adapter (the VIC-1011A) that plugged into the user port to convert these TTL signals to true RS-232 voltage levels for connection to modems or serial printers. The presence of full modem control lines (RTS, CTS, DCD, etc.) on the user port was quite powerful – it meant the VIC-20 could interface with serial devices in a way PCs typically did via dedicated UART chips. The VIA's CB1/CB2 could even be

used in a shift register mode to assist with serial framing, but due to the mentioned silicon bug the ROM routines did bit-banging instead. Regardless, hobbyists could and did create custom serial links with just a few transistors for level shifting.

In summary of pinout: the user port offers a mix of pure digital I/O (8 bits + 2 handshake) and some special purpose lines (reset, power, ground, and mirror signals for joystick/cassette/serial). All these signals are directly tied into the VIC-20's core logic, so misuse (short circuits, overvoltage) could crash or damage the computer – a risk and freedom reminiscent of today's microcontroller boards where you can connect anything, but you must be mindful of electrical limits.

Supported Protocols and Uses: The flexibility of the user port allowed it to support several kinds of connectivity: - **Serial Communication (RS-232/V.24):** As noted, by using the PB0/PB1/etc lines for Rx/Tx/handshake and the VIA's timers, the VIC-20 could communicate with serial devices. For example, the **VICModem**, a 300 baud modem released by Commodore, plugged directly into the user port (no cable needed) and used those lines to send/receive data over telephone lines ⁴⁶. Users ran terminal software (e.g. VIC-Term) to connect to bulletin board systems. Baud rates were typically 300 or 1200; higher rates (e.g. 2400) were theoretically possible but pushing the limits of what the 1 MHz 6502 could bit-bang reliably. Another serial use was connecting to other computers or instruments – the user port made the VIC-20 a cheap serial terminal or controller. It's important to emphasize the **voltage levels**: the user port signals are 0–5 V TTL, so direct connection to RS-232 devices (which use +/-12 V levels) required an interface. With the proper interface (level shifter IC or simple transistor network), the VIC-20's user port fully implemented the RS-232 protocol lines ⁴⁴. - **Parallel I/O (Centronics/Printer or GPIO):** The 8 data lines could be used as a parallel output port, similar to a printer port. In fact, Commodore's documentation mentions the user port being usable for **Centronics parallel printers** ⁴⁷. A cable or adapter could connect those 8 lines to a printer's parallel input, and software would strobe the appropriate handshake via one of the CB lines. Some third-party interfaces did exactly this, allowing the VIC-20 (and later C64) to print to standard parallel printers instead of Commodore's serial IEC printers. More generally, the user port could interface with custom hardware: one could attach, say, an array of LEDs to display binary output, or read 8 switches in parallel. Many electronics hobby projects for the VIC-20 involved the user port – for instance, controlling relays or reading sensors. Because you could individually set each line as input or output, it was effectively **general-purpose I/O (GPIO)**. Modern comparisons are apt: hooking up a VIC-20 user port to electronics is much like using an Arduino's digital pins, except the programming is done via PEEKs and POKEs to the VIA registers. One difference is timing – without dedicated PWM or analog, any precise control (like generating an exact frequency pulse) had to be done in software with cycle counting or VIA timers. Enthusiasts have used the user port for things like home automation (turning lights on/off via relays) or even reading analog sensors (by timing a capacitor's charge on an input pin, since no ADC present). - **Networking and Other Interfaces:** Beyond printers and modems, the user port saw some creative uses. There was an **IEEE-488 parallel bus adapter (VIC-1112)** that connected to the user port, effectively giving the VIC-20 a full IEEE-488 interface to talk to older PET-series disk drives or lab equipment. This worked because the user port had enough lines to implement the 8-bit data and handshake lines of IEEE-488 (though not all lines, it had to multiplex some). Another use was **simple networking**: hobbyists connected two Commodore machines via their user ports (with a custom cable) to achieve a direct parallel link (like a null-printer cable). Software could then transfer data much faster than over serial. In modern terms, it's akin to a GPIO-to-GPIO communication between two microcontrollers. The VIC-20 user port also could host custom cartridges that didn't need the full memory expansion port – for example, some reset buttons or diagnostic dongles were made for the user port (since it has a reset pin and power). - **GPIO for DIY Electronics:** Commodore even published "Easy Interface" project guides, suggesting experiments like hooking up the VIC-20 to external

counters, sensors, etc., through the user port. Because the VIA also includes timers and can generate interrupts on input changes, fairly sophisticated control systems could be built. For example, one could measure the frequency of an external pulse train by configuring a CB1 line to trigger an interrupt on each pulse and timing it via the VIA's counter – essentially using the VIC-20 as a data acquisition device. In the 1980s some did build alarm systems or robot controllers with VIC-20s, leveraging this port.

Interfacing and Modern Analogies: To use the user port, one typically built a **simple interface circuit** on a card-edge connector. For instance, an RS-232 interface might include level translator chips (like the MAX232 or in those days, just transistors/diodes) to convert TTL to +/-12 V. A relay board might use transistors or opto-isolators driven by the port bits to switch higher voltages. In terms of *electrical specs*, all signals are standard 5 V TTL, meaning logic high ≈ 5 V, logic low ≈ 0 V, and any input must also be 0–5 V (anything beyond could damage the VIA). The port's 5 V supply being limited to 100 mA is analogous to how a Raspberry Pi GPIO header can only supply a limited current – enough for small sensors or ICs, but not for motors or large displays without external power.

A modern engineer might compare the VIC-20 user port to an **Arduino's GPIO plus a UART** all present on one connector. It required the user's software to implement protocols – for example, to talk to a parallel printer, you'd bit-bang the handshake (set data lines, toggle a strobe line, etc.) similar to how one might in firmware on a microcontroller. Unlike modern PCs which have abstracted ports (USB, etc.), the VIC-20 exposed raw lines. This was simultaneously empowering and challenging. Many educational books of the era taught users how to POKE the VIA registers to make LEDs blink or read digital inputs – essentially teaching low-level hardware interfacing. This aspect of the VIC-20 (and its successor, the C64, with a similar user port) made it a popular machine for hardware hacking.

One caveat: The VIC-20's user port **is not pin-compatible with the C64's user port**, despite looking similar. Commodore rearranged some lines on later machines. So peripherals had to be designed specifically for VIC-20 vs C64 (the voltage and concept remained the same though).

Applications of the User Port: In practice, the VIC-20 user port saw use in: - **Telecommunications:** The VICModem (for dialing into BBSes) and other third-party modems. This turned the VIC-20 into a terminal for calling information services – a big deal in the early 1980s, as it was one of the cheapest ways to get online. - **Printing:** User port printer interfaces allowed connection to Centronics parallel printers, which were faster and more common than Commodore's serial printers. Businesses or home users used VIC-20s for word processing with such printers. - **Education and Experimentation:** Schools or hobbyists connected interface boards for controlling laboratory experiments, similar to how one might use a modern microcontroller in a physics lab. The VIC-20 could log temperature from a simple analog-to-digital setup (e.g., using the cassette port's analog line in creative ways, or a voltage-to-frequency converter feeding a user port counter). Its timers and interrupts made it feasible to do rudimentary data logging. - **Games and Peripherals:** A few game or utility cartridges used the user port – for example, a “reset button” cartridge (just grounds the reset pin via a pushbutton) could plug in there. Some sound or speech synthesizer modules connected via user port as well, using the port to output audio data (the Magic Voice speech module for C64 had a VIC-20 variant). - **Custom Controllers:** There are anecdotes of people controlling model trains, robots, or homebrew analog synthesizers with the VIC-20 user port. With some DAC (digital-to-analog converter) connected to the 8-bit port, the VIC-20 could output crude analog signals (e.g., generate a waveform for sound or control).

All of this capability in the user port underscores a design philosophy: the VIC-20 was not only a game machine but also a **hobbyist's playground**. It bridged the gap between the solder-their-own computer kits of the 1970s and the closed appliances of later years. For a modern comparison, it's as if your home computer had an integrated Arduino-like interface – something quite rare now when interfaces tend to be high-level (USB, Bluetooth, etc.).

In conclusion, the Commodore VIC-20's technical architecture – with its 6502 CPU, minimal memory, VIC graphics/sound chip, and VIA-controlled I/O – represents a **clean, straightforward 8-bit computer design**. Its performance was modest, but well-balanced for the era, and it introduced many people to computing by exposing the workings (one could directly manipulate memory and I/O ports to see immediate effects). Modern experts in computer architecture can appreciate the VIC-20 as a study in efficient use of scant resources: the entire system had to operate with a few kilobytes and a 1 MHz clock, yet it achieved interactive graphics, sound, and expandability. Many concepts from the VIC-20 remain relevant – from memory-mapped I/O (used in today's SoCs and microcontrollers) to the idea of cycle timing and bus arbitration (seen in how GPUs/DMA's interact with CPUs today). And just as the VIC-20's user port provided a gateway to external hardware, modern embedded computers continue that tradition with GPIO and serial interfaces. In short, the VIC-20 is a charming early example of computer engineering trade-offs, one that modern computing professionals can look back on as a foundational step in the evolution of personal computing ⁴⁸ ⁴⁹ .

Sources:

- Zimmers.net VIC-20 technical summary (specifications for CPU, memory, video, sound, ports) ⁵⁰ ⁷
- C64-Wiki (VIC-20 article) – details on hardware specs (CPU clock, RAM/ROM breakdown, graphics modes, sound channels, I/O interfaces) ⁴ ⁸
- VIC-20 Reloaded – Memory map and I/O addresses (mapping of VIA chips, user port bits and RS-232 signals, memory layout) ⁵¹ ⁵²
- Retrocomputing StackExchange – discussion on VIC-20 VIA usage (explains which VIA handles user port vs keyboard, confirming two VIA design) ⁵³ ¹⁶
- Retro Isle – VIC-20 Ports and Pinouts (user port pin assignment with signals like +5V, ground, JOY signals, and description of VIA Port B usage) ³³ ¹⁷
- “Instructions per second” computing wiki – reference MIPS for 6502 and Z80 CPUs ² (to compare relative CPU performance)
- C64-Wiki and Commodore user manual excerpts – details on user port capabilities (noting RS-232/Centronics uses) ⁴⁷ and power limits ³³ .

¹ VIC-20 - Wikipedia

https://en.wikipedia.org/wiki/Commodore_VIC-20

² Instructions per second | Computer Wiki | Fandom

https://computer.fandom.com/wiki/Instructions_per_second

³ Comparing raw performance of the Z80 and the 6502 - Retrocomputing Stack Exchange

<https://retrocomputing.stackexchange.com/questions/5748/comparing-raw-performance-of-the-z80-and-the-6502>

⁴ ⁵ ⁸ ¹¹ ²⁵ ²⁷ ²⁹ ³⁰ ⁴⁷ ⁴⁹ VIC-20 - C64-Wiki

<https://www.c64-wiki.com/wiki/VIC-20>

6 14 15 18 19 20 21 43 44 45 51 52 **Commodore VIC-20 Memory Map - VIC-20 Reloaded**

<https://vic20reloaded.com/commodore-vic-20-memory-map/>

7 10 13 26 48 50 **Commodore VIC-20 Computer : The Friendly Web Page**

<https://www.zimmers.net/cbmpics/cvic.html>

9 16 22 23 24 53 **hardware - Why does the Vic-20 have two VIA chips? - Retrocomputing Stack Exchange**

<https://retrocomputing.stackexchange.com/questions/17353/why-does-the-vic-20-have-two-via-chips>

12 **Shift/Run Stop VIC 20 and C64 differences. - Denial**

<http://sleepingelephant.com/ipw-web/bulletin/bb/viewtopic.php?t=5364>

17 28 31 32 33 34 35 36 37 38 39 40 41 42 **Retro Isle - VIC-20 Technical Section**

<http://www.retroisle.com/commodore/vic20/Technical/Hardware/VIC20ports.php>

46 **Cartridge modems - was the Vic-20 a special case?**

<https://retrocomputing.stackexchange.com/questions/7309/cartridge-modems-was-the-vic-20-a-special-case>